TRANSURBAN — Demonstração completa da API no auditório

Guia de falas e testes ao vivo • 20/10/2026 • Sem slides

Equipe: José Henrique, Bruno, Kauan, Gustavo, Ana Carolina, Jullyany e Camila. José Henrique pode operar todos os cliques. Bruno e Kauan mantêm as falas curtas. DS e IA estão separados em duas partes neste documento; os roteiros individuais continuam disponíveis. Reserve aproximadamente 14 a 18 minutos com cliques, ou ajuste ao tempo autorizado pela escola.

O que mostrar ao público

A apresentação é da API funcionando no Swagger. Este documento orienta a equipe; não substitui a execução. O site é um complemento opcional e não é necessário para demonstrar DS e IA.

No computador que estiver executando o servidor, abra:

```
http://127.0.0.1:8000/docs
```

Em cada operação: abra a rota, clique Try it out, preencha os dados quando necessário e clique Execute. Aguarde Server response e só então explique o resultado. Example Value não é evidência de execução.

Preparação do operador, antes da apresentação

Extraia o pacote somente API e site. No VS Code, abra a pasta TRANSURBAN_API_SITE, onde está main.py. Requer Python 3.12 e internet para a primeira instalação. Não execute de dentro do ZIP. Se usar o pacote antigo com materiais, a pasta equivalente é backend_escola.

No Windows, execute um comando de cada vez:

```
py -3.12 iniciar_apresentacao.py --preparar
py -3.12 iniciar_apresentacao.py --testar
py -3.12 iniciar_apresentacao.py --diagnostico
py -3.12 iniciar_apresentacao.py
```

No Linux, primeiro confira que python3 --version indica 3.12:

```
python3 iniciar_apresentacao.py --preparar
python3 iniciar_apresentacao.py --testar
python3 iniciar_apresentacao.py --diagnostico
python3 iniciar_apresentacao.py
```

Depois de Application startup complete, mantenha o terminal aberto. O inicializador usa SQLite local e preserva o .env: não precisa de senha Supabase. Não anuncie conexão remota que não foi comprovada. Se a porta 8000 estiver ocupada, use iniciar_apresentacao.py --port 8002 e troque a porta nos endereços.

Antes do público entrar, execute GET /health: esperado status ok e banco sqlite_local. Confirme zoom legível no telão e projeção da mesma tela. Não exiba .env, senhas ou autenticações. Depois da instalação, o Swagger e a análise do CSV local não precisam de internet; mapa, clima e cálculo de vias do site precisam.

Validação da entrega: 41 testes da suíte completa passaram em Linux, além do diagnóstico e do ensaio de duas contas do site. Esse número não é uma contagem exclusiva de DS ou de IA. Windows, GPS físico e projetor precisam ser ensaiados no equipamento real.

Sequência rápida dos testes

Etapa	Operação	Resultado esperado
Antes da fala	GET /health	200, status ok e banco sqlite_local.
DS — cadastro	POST /cidades	201; anote o id_cidade retornado.
DS — vínculo	POST /linhas	201; use o ID real da cidade.
DS — consulta	GET /cidades e GET /linhas	200; o cadastro está na lista, com o nome da cidade.
DS — proteção	POST /linhas com cidade ausente	404; tentativa não salva. Confira antes que o ID não existe.
DS — validação	POST /cidades com nome só de espaços	422; entrada rejeitada.
IA — origem	GET /ia/dados-publicos/fonte	Fonte comunitária histórica de Curitiba e período declarado.
IA — qualidade	GET /ia/dados-publicos/qualidade	Auditoria do CSV, identificadores e hashes.
IA — resumo	GET /ia/dados-publicos/resumo	71.086 registros; 309 códigos; média 230,0518; mediana 162.
IA — comparação	GET /ia/dados-publicos/agrupamentos/{dimensao}	dimensao linha ou servico; parâmetro limite = 5 para o telão.

Leia somente os parágrafos FALA. NA TELA e APOIO orientam o operador e as perguntas. Não anunciem resultado antes de ele aparecer na resposta real.

Abertura geral — José Henrique

FALA: “Bom dia, pessoal! Eu sou o José Henrique e esta é a nossa equipe: Bruno, Kauan, Gustavo, Ana Carolina, Jullyany e Camila. Vamos apresentar o TRANSURBAN, um projeto acadêmico pensado para a mobilidade em Maringá e Sarandi.

FALA: Hoje vamos mostrar a API: a parte que recebe um pedido, verifica as informações, trabalha com os dados e devolve uma resposta.

FALA: A apresentação terá duas partes. Em DS, vamos cadastrar e consultar uma cidade e uma linha, mostrando a ligação entre as tabelas e o tratamento de entradas inválidas. Em IA, vamos analisar um CSV e explicar os indicadores que a fonte permite produzir.

FALA: Os cadastros da demonstração usam banco local. O código DEMO-20 é apenas um exemplo de teste. Na análise, utilizamos programação pública histórica de Curitiba, de origem comunitária: não são atrasos medidos de Maringá ou Sarandi.

FALA: O Bruno vai começar explicando o objetivo dos cadastros.”

PARTE 1 — DS: cadastros, relacionamento e validações

2. Bruno — objetivo dos cadastros, fala curta

FALA: “Nosso sistema organiza cidades e linhas de ônibus. Primeiro cadastramos a cidade. Depois, cadastramos uma linha ligada a ela. Assim conseguimos consultar a linha e saber a qual cidade ela está vinculada. Agora o Kauan vai explicar a consulta.”

APOIO: Bruno não precisa operar o computador nem explicar bibliotecas. Pode falar com calma, em aproximadamente 20 segundos.

3. Kauan — o que é GET, fala curta

FALA: “Consultar é pedir para ver uma informação que já está salva. Na API, usamos GET para isso. Ele mostra os dados sem fazer um novo cadastro. O José Henrique vai mostrar como cadastramos e consultamos na prática.”

APOIO: Kauan não precisa executar comandos. A fala pode durar aproximadamente 20 segundos.

4. José Henrique — organização dos arquivos

NA TELA: mostre brevemente os quatro arquivos no VS Code, sem percorrer todo o código.

FALA: “Organizamos o código em arquivos com responsabilidades diferentes. O database.py configura a conexão com o banco. O models.py define as tabelas e os relacionamentos. O schemas.py valida os dados recebidos e organiza as respostas. O main.py disponibiliza as rotas da aplicação.

FALA: Usamos Python com FastAPI para a API. O SQLAlchemy trabalha com o banco de dados, e o Pydantic faz as validações. Essa separação facilita entender o código e encontrar onde uma alteração deve ser feita.”

APOIO: uma sessão de banco é fornecida para a operação e depois encerrada. commit confirma a transação; rollback desfaz uma transação que falhou. Não é necessário falar esses detalhes se o professor não perguntar.

5. José Henrique — cadastrar a cidade

NA TELA: volte ao Swagger. Abra POST /cidades → Try it out. Envie:

```
{"nome":"Sarandi"}
```

NA TELA: clique Execute. Espere 201 e anote o id_cidade que realmente apareceu. Não assuma que é 1. Se já existem cadastros do ensaio, os IDs podem ser outros.

FALA: “Agora cadastramos Sarandi. POST é o método que usamos para enviar esse cadastro. O código 201 indica que o registro foi criado.

FALA: O campo id_cidade é a identificação da cidade no banco. Ele é a chave primária: permite identificar cada registro de forma única.”

APOIO: no SQLite deste recorte, nome da cidade e código da linha não têm restrição UNIQUE. Repetir o cadastro pode criar outro registro com outro ID. Não prometam erro de duplicidade que esta configuração não implementa. Isso é diferente de outros esquemas PostgreSQL que possam ter restrições adicionais.

6. José Henrique — cadastrar a linha e explicar a chave estrangeira

NA TELA: abra POST /linhas → Try it out. Substitua o 1 abaixo pelo id_cidade realmente retornado:

```
{
  "codigo": "DEMO-20",
  "nome": "Linha da apresentação",
  "id_cidade": 1
}
```

NA TELA: clique Execute. Espere 201. Aponte id_cidade e o nome da cidade na resposta.

FALA: “Agora cadastramos uma linha de demonstração usando a identificação da cidade. O campo id_cidade faz a ligação entre as duas tabelas: ele é a chave estrangeira.

FALA: Neste recorte do sistema, uma cidade pode ter várias linhas cadastradas, e cada linha aponta para uma cidade existente. A API verifica essa existência antes de salvar.

FALA: Essa relação evita deixar uma linha apontando para um registro de cidade que não existe.”

APOIO: esta relação é o recorte cadastral da avaliação. Não representa, sozinha, todos os municípios atravessados por uma linha intermunicipal ou todas as paradas de seu itinerário.

7. José Henrique — consultar os registros

NA TELA: execute GET /cidades e depois GET /linhas. Espere 200. Localize o registro criado; não suponha que a lista terá somente um item.

FALA: “Agora consultamos o que acabamos de salvar. O código 200 indica que a consulta foi atendida. A linha aparece acompanhada do nome da cidade, facilitando a leitura da informação.

FALA: Esse formato de campos e valores é chamado JSON. Uma interface pode usar essa resposta para mostrar os dados ao usuário. O Gustavo vai testar o que acontece quando as informações estão erradas.”

8. Gustavo — cidade inexistente, erro esperado 404

NA TELA: confira por GET /cidades que o ID 999999 não existe. Em POST /linhas, envie:

```
{
  "codigo": "TESTE-ERRO",
  "nome": "Tentativa inválida",
  "id_cidade": 999999
}
```

FALA, antes de clicar: “Até aqui usamos dados válidos. Agora vamos informar uma cidade inexistente, de propósito, para testar a validação.”

NA TELA: clique Execute. Esperado: 404 e detail: Cidade não encontrada. Se esse ID existir, escolha outro ID positivo comprovadamente ausente.

FALA: “A API identificou que a cidade não existe e recusou o cadastro. O 404 é o resultado esperado deste teste.”

NA TELA: execute GET /linhas. Confirme que essa tentativa não criou um novo registro TESTE-ERRO. Compare com a lista anterior se já havia testes antigos.

FALA: “A consulta confirma que essa tentativa inválida não foi salva.”

9. Gustavo — nome vazio, erro esperado 422

NA TELA: em POST /cidades, envie exatamente um nome com espaços:

```
{"nome":"   "}
```

NA TELA: clique Execute. Esperado: 422. O texto técnico da resposta pode aparecer em inglês; não precisa ser lido inteiro.

FALA: “Agora enviamos um nome formado apenas por espaços. A validação rejeitou a entrada e retornou 422. Isso evita guardar uma cidade sem nome.

FALA: Os dois erros foram provocados de propósito. Eles mostram que o sistema verifica as entradas e devolve respostas para orientar quem está usando a API.”

10. José Henrique — persistência, se houver tempo

NA TELA: memorize o id_linha criado. No terminal da API, pressione Ctrl+C. Inicie novamente com o mesmo comando usado antes, sem --preparar. Aguarde Application startup complete e execute GET /linhas.

FALA: “Encerramos apenas o servidor e iniciamos novamente. O cadastro continua no banco local. Isso mostra a persistência dos dados: eles não ficam guardados apenas na tela ou na memória do programa.”

APOIO: não apague apresentacao_local.db e não rode uma configuração de banco diferente. Se o tempo estiver curto, explique a persistência e mantenha esse teste como apoio para perguntas.

11. Ana Carolina — testes, GitHub e conclusão

FALA: “Além da demonstração, usamos testes automatizados e registramos evidências das respostas da API. Isso ajuda a conferir o funcionamento depois de alterações no código.

FALA: O código está versionado no GitHub, permitindo acompanhar as mudanças e compartilhar a entrega.

FALA: Nesta apresentação, mostramos a organização do backend, o cadastro e a consulta de duas entidades relacionadas, a validação das entradas e os testes de situações de erro.

FALA: O site pode utilizar essas respostas para apresentar as informações visualmente. Aqui, nosso foco foi demonstrar que a API recebe os pedidos, aplica as regras e trabalha com o banco de dados. Com isso, concluímos a parte de DS. Agora a Jullyany e a Camila vão apresentar a análise do CSV.”

Apoio para perguntas dos professores

Pergunta	Resposta curta
API e site são a mesma coisa?	A API recebe pedidos e devolve dados. O site apresenta telas e utiliza esses dados. O mesmo servidor pode servir ambos.
O que é uma rota?	Um endereço da API associado a uma operação, como GET /linhas.
Qual a diferença entre POST e GET?	POST envia o cadastro; GET consulta informações. Neste recorte, GET não cria registros.
PK e FK significam o quê?	Chave primária identifica o registro; chave estrangeira liga esse registro a outra tabela.
Qual a relação entre cidade e linha?	Neste recorte, uma cidade pode ter várias linhas; cada linha aponta para uma cidade cadastrada.
Por que aparece o nome da cidade na linha?	A consulta usa o relacionamento entre as tabelas para produzir uma resposta mais compreensível.
Onde os dados ficam?	Nesta demonstração, em um arquivo SQLite local. Não estamos demonstrando gravação remota no Supabase.
O que o ambiente virtual faz?	Isola as bibliotecas Python necessárias ao projeto.
O que significam 200, 201, 404 e 422?	Consulta atendida; registro criado; cidade não encontrada; entrada inválida.
O banco impede nomes repetidos?	Não neste modelo SQLite atual. Cada novo registro recebe outro ID. Não confundir com esquemas remotos que tenham UNIQUE.
O que falta para uso operacional?	Integrações, autenticação operacional, regras e infraestrutura precisam ser validadas para cada função. Ter uma tela não comprova uma integração.

Se algo falhar no auditório

Se 127.0.0.1 recusar conexão, confira se o servidor continua aberto no mesmo computador. Se o terminal mostrar erro, não anuncie sucesso. Não tente corrigir senhas ou trocar de banco no palco.

FALA DE APOIO: “A execução não respondeu como esperado neste momento. Temos evidências de uma execução anterior e vamos identificá-las como evidências, sem confundir com o teste ao vivo.”

Depois da apresentação, Ctrl+C encerra o servidor. Saia do WhatsApp Web se o computador for compartilhado.

PARTE 2 — IA: CSV, qualidade e indicadores

1. Jullyany — abertura

FALA: “Agora vamos apresentar a parte de Ciência de Dados. Eu sou a Jullyany, e vou dividir essa explicação com a Camila.

FALA: Nosso objetivo foi organizar uma fonte de dados, verificar sua qualidade, produzir indicadores e disponibilizar os resultados pela API.

FALA: Para isso, usamos Python e a biblioteca Pandas. Vamos mostrar o arquivo utilizado, explicar os cálculos e consultar os resultados ao vivo.”

2. Jullyany — origem e período da fonte

NA TELA: abra GET /ia/dados-publicos/fonte → Try it out → Execute. Mostre cidade, feed_oficial, publicador_gtfs e periodo_declarado_feed. Não precisa ler a URL ou o hash inteiro em voz alta.

FALA: “O TRANSURBAN é um projeto voltado a Maringá e Sarandi. Porém, não tínhamos uma base local de atrasos reais disponível para essa análise.

FALA: Por isso, utilizamos uma base pública histórica de programação do transporte de Curitiba, disponibilizada em um GTFS comunitário. GTFS é um formato usado para organizar informações de transporte público.

FALA: Esse conjunto é comunitário e não é uma publicação oficial da URBS. O período declarado vai de 28 de setembro a 5 de outubro de 2026.

FALA: Estamos estudando uma programação publicada. Não estamos mostrando rastreamento dos ônibus nem comprovando que todas as viagens aconteceram.”

APOIO: o publicador comunitário declara origem em dados URBS espelhados pelo C3SL/UFPR; a equipe conferiu o derivado publicado, o código do conversor e a catalogação na Mobility Database. Não diga que a equipe coletou GPS ou obteve certificação da URBS. O período é o declarado no feed, não uma lista de datas de operação observada.

3. Jullyany — o que há no CSV

NA TELA: se necessário, mostre apenas o cabeçalho de data/dados_publicos/programacao_curitiba.csv no VS Code. Não abra milhares de linhas durante a fala.

FALA: “O CSV é um arquivo organizado como uma tabela. As colunas representam informações, e cada linha representa um registro.

FALA: Nosso arquivo tem o identificador de programação, o código e o nome da linha, o tipo de serviço, a operadora e informações sobre cidade e período da fonte.

FALA: Cada linha do arquivo representa um registro de programação publicado. Não significa um passageiro, um ônibus diferente ou uma viagem comprovadamente realizada.

FALA: Também podemos dizer que esse CSV é desnormalizado: ele reúne informações relacionadas em um mesmo arquivo. Por exemplo, o nome da linha e a operadora aparecem junto de cada registro de programação. Isso facilita certas análises, mas exige conferir a consistência das informações repetidas.”

APOIO — dicionário das colunas:

Coluna	Significado e cuidado
id_programacao	Identificador do registro, derivado do trip_id publicado; lido como texto.
codigo_linha	Código da linha; texto, para não perder zeros ou interpretar como medida numérica.

Coluna	Significado e cuidado
nome_linha	Nome associado ao código da linha.
tipo_servico	Categoria: dias_uteis, sabado ou domingo. Não prova operação em uma data específica.
operadora	Informação associada à agência do feed; não prova validação oficial da publicação comunitária.
cidade_fonte e uf_fonte	Curitiba e PR; não são Maringá e Sarandi.
periodo_feed_inicio e periodo_feed_fim	Período declarado pelo feed, não datas de medições de atraso.

O arquivo tem nove colunas; a tabela agrupa alguns pares de colunas para facilitar a explicação. Todas são inicialmente lidas como texto. As medidas numéricas são calculadas a partir das contagens.

4. Jullyany — Pandas e qualidade dos dados

NA TELA: volte ao Swagger e execute GET /ia/dados-publicos/qualidade. Aponte registros_lidos, ausentes_por_coluna, ids_duplicados, hash_csv_confere e apta_para_calculos_do_snapshot.

FALA: “Usamos o Pandas para ler e organizar o CSV. Antes de calcular, conferimos se as colunas esperadas estão presentes, se existem campos vazios ou identificadores repetidos e se o mesmo código aparece com nomes divergentes.

FALA: Também verificamos as categorias de serviço e se a cidade e o período continuam compatíveis com a fonte documentada.

FALA: Outro cuidado foi conferir a integridade dos arquivos por meio de um hash. Ele funciona como uma assinatura do conteúdo: se o arquivo mudar, a assinatura também muda.

FALA: Se essas verificações encontrarem uma divergência, a API bloqueia os indicadores para revisão. Não descartamos ou substituímos registros silenciosamente.

FALA: Agora a Camila vai mostrar as perguntas que conseguimos responder e os indicadores produzidos.”

APOIO: não fale que todos os campos são perfeitos apenas porque um indicador apareceu. Mostre os resultados reais da auditoria. Não confunda o hash com certificação de verdade do conteúdo: ele detecta alteração de bytes, não comprova sozinho que a programação foi cumprida.

5. Camila — perguntas e contagens

NA TELA: execute GET /ia/dados-publicos/resumo. Mostre registros_programacao, linhas_distintas e tipos_servico_distintos.

FALA: “Depois de entender e verificar a fonte, formulamos perguntas que esses dados conseguem responder. Quantos registros de programação existem? Quantas linhas aparecem? Como esses registros estão distribuídos entre as linhas e as categorias de serviço?

FALA: Nosso conjunto possui 71.086 registros de programação e 309 códigos de linha distintos, em três categorias de serviço.

FALA: Esses números descrevem o conteúdo do arquivo. Eles não representam passageiros transportados, ônibus diferentes ou viagens comprovadamente realizadas.

FALA: Definir corretamente o que estamos contando é uma parte essencial da análise.”

6. Camila — soma, média e mediana

NA TELA: continue no resumo. Aponte soma_registros_nos_grupos_linha, media_registros_programacao_por_linha e mediana_registros_programacao_por_linha.

FALA: “Ao somar as contagens dos grupos de linha, voltamos ao total de 71.086 registros. Essa conferência ajuda a verificar a consistência do agrupamento.

FALA: A média é aproximadamente 230,05 registros por linha. Para calculá-la, dividimos o total de registros pela quantidade de linhas.

FALA: A mediana é 162. Ela é encontrada depois de ordenar as quantidades e identificar o valor central.

FALA: A média pode ser influenciada por linhas com quantidades muito maiores. A mediana ajuda a observar o centro da distribuição. Por isso, as duas medidas oferecem uma visão mais completa.

FALA: Esses indicadores consideram o conjunto de categorias de serviço do arquivo. Não são médias diárias, minutos de atraso nem viagens efetivamente realizadas.”

APOIO: 71.086 dividido por 309 resulta em aproximadamente 230,0518. A mediana é calculada sobre as 309 contagens por código de linha; ela não é a mediana dos códigos numéricos.

7. Camila — agrupamento por linha

NA TELA: abra GET /ia/dados-publicos/agrupamentos/{dimensao}. Em dimensao, selecione linha. No parâmetro chamado limite, preencha 5. Clique Execute. O nome correto do parâmetro é limite, não limit.

FALA: “Nesta consulta, agrupamos os registros por linha. Agrupar significa reunir registros que têm uma característica em comum e calcular informações para cada grupo.

FALA: Isso permite comparar a distribuição da programação entre as linhas presentes no arquivo.

FALA: Estamos mostrando cinco grupos para facilitar a leitura no telão. O limite reduz a lista exibida, mas não transforma cinco no total da base. A resposta identifica o total de grupos e a quantidade que está sendo mostrada.”

APOIO: leia total_grupos, grupos_exibidos e resultados. soma_registros_nos_grupos é o total completo; soma_registros_dos_grupos_exibidos corresponde somente à amostra exibida. Não troque essas duas somas.

8. Camila — agrupamento por serviço

NA TELA: na mesma operação, selecione dimensao = servico e mantenha limite = 5, ou abra o endereço /ia/dados-publicos/agrupamentos/servico?limite=5. Execute.

FALA: “Também agrupamos os dados por categoria de serviço: dias úteis, sábado e domingo.

FALA: Essa separação permite comparar a composição da programação publicada. Ainda assim, estamos descrevendo o arquivo, e não confirmando o que aconteceu nas ruas.”

APOIO — valores do snapshot, leia somente se aparecerem:

Categoria	Registros de programação
dias_uteis	33.425
sabado	20.754
domingo	16.907
Total	71.086

Não interprete esse total como viagens de uma semana específica ou como número de passageiros. Os rótulos de categoria, sozinhos, não comprovam operação em cada data.

9. Camila — API, JSON e informação indisponível

NA TELA: volte a GET /ia/dados-publicos/resumo. Aponte viagens_realizadas e atraso_medio_min, ambos null.

FALA: “Os resultados são disponibilizados pela API em JSON. É um formato organizado que pode ser utilizado por um site ou dashboard para construir cartões, tabelas e gráficos.

FALA: Tivemos o cuidado de não calcular informações que a fonte não permite. O atraso médio aparece como null porque não temos medições reais suficientes para produzir esse indicador. Isso significa informação indisponível, e não atraso zero.

FALA: Nossa análise não prevê quando um ônibus vai chegar e não identifica causas de atraso. Ela descreve a programação do arquivo.”

10. Jullyany e Camila — conclusão

JULLYANY — FALA: “Com esse trabalho, percorremos as etapas de uma análise de dados: entender a fonte, organizar o CSV, verificar a qualidade, formular perguntas e calcular indicadores.”

CAMILA — FALA: “A API disponibiliza esses resultados para uma próxima etapa de dashboard. Nosso cuidado foi apresentar informações sustentadas pelos dados disponíveis, deixando claras a origem, a unidade de contagem e as limitações. Obrigada pela atenção!”

Apoio: como explicar os códigos de IA

Arquivo ou função	Explicação simples
importar_dados_publicos.py	Converte o conjunto GTFS publicado para o CSV da análise, preservando origem e identificadores.
dados_publicos.py	Lê, verifica e analisa o CSV; contém as rotas /ia/dados-publicos.
pd.read_csv	Lê o arquivo em uma tabela do Pandas.
dtype='string'	Preserva os campos como texto, inclusive códigos e identificadores.
groupby	Reúne registros pela dimensão escolhida, como código de linha ou tipo de serviço.
size	Conta quantos registros existem em cada grupo.
sum	Soma as contagens dos grupos. Não soma códigos de linha.
mean e median	Calculam média e mediana das contagens.
hashlib.sha256	Produz o hash usado na verificação de integridade dos arquivos.
FastAPI	Disponibiliza os resultados por rotas HTTP, devolvendo JSON.

A base apresentada é programacao_curitiba.csv. analise.py, ia_routes.py e atrasos_analise.csv pertencem à base sintética anterior de testes. Não usem os atrasos sintéticos como medições reais no palco.

Perguntas que podem aparecer

Pergunta	Resposta curta
Por que Curitiba?	Não tínhamos uma base local de atrasos reais disponível. Utilizamos uma fonte pública histórica e documentamos essa diferença.
A base é oficial?	Não. É um derivado comunitário publicado em GTFS. A origem declarada e as limitações estão documentadas.
Os dados são inventados?	Esta análise utiliza registros publicados de programação. A base sintética anterior fica separada para testes e não é anunciada como dado real.
Vocês mediram os atrasos?	Não. Não há medições suficientes nessa fonte para calcular atraso real.
O que é null?	Informação indisponível; não significa zero.
Qual a diferença entre média e mediana?	Média usa todos os valores na divisão; mediana é o valor central depois de ordenar as contagens.
Onde entra o Python?	Na conversão, leitura, verificação e análise dos dados, além da API.
Onde entra o Pandas?	No trabalho com a tabela CSV, contagens e agrupamentos.
Vocês treinaram uma IA?	Não. Nesta avaliação, trabalhamos Ciência de Dados: preparação, análise e disponibilização dos indicadores.
Por que não agrupar por data de viagem?	As datas disponíveis são do período declarado do feed, não datas de operação observada de cada registro. Não inventamos uma série diária.
O que vem depois?	Utilizar os resultados no dashboard e buscar fontes locais adequadas às perguntas que ainda não podem ser respondidas.

Referências para conferência

Publicação comunitária: https://github.com/benaytms/urbs-gtfs

Metadados, período, URLs e hashes desta cópia: backend_escola/data/dados_publicos/fonte.json. O arquivo registra também a catalogação na Mobility Database e a versão do código usada na conferência. Consulte a operação /ia/dados-publicos/fonte durante a apresentação.

Se uma consulta falhar

Confira se o servidor permanece aberto no mesmo computador. Não invente um resultado nem apresente um JSON salvo como se tivesse acabado de ser calculado.

FALA DE APOIO: “A consulta não respondeu como esperado neste momento. Temos evidências de uma execução anterior e vamos identificá-las como evidências, sem confundir com o teste ao vivo.”

O roteiro é apoio para compreender e ensaiar. Se o professor fizer uma pergunta fora do que foi verificado, digam que precisam conferir a fonte ou o código antes de responder.

Encerramento geral — José Henrique

FALA: “Mostramos os pedidos chegando à API, as regras sendo aplicadas e os resultados sendo devolvidos. Em DS, verificamos cadastro, consulta, relacionamento e validação. Em IA, explicamos a fonte, conferimos o CSV e consultamos os indicadores. Os próximos passos dependem de fontes e integrações verificadas. Obrigado pela atenção!”

Se pedirem para ver o site

Abra http://127.0.0.1:8000/app/ no mesmo computador. Use uma janela normal e outra anônima para contas de passageiro e motorista separadas. Demonstre somente o que já foi ensaiado: início da viagem, embarque/desembarque, relato de lotação e avaliação. GPS depende de permissão e sinal.

FALA: “Este é um protótipo funcional da interface. As contas e avaliações são salvas no nosso banco local; as viagens são autodeclaradas. O mapa usa dados geográficos reais, mas o percurso escolhido não é um itinerário oficial da operadora. Pagamentos e vale-transporte são demonstrações, sem movimentar dinheiro.”

A tarifa de referência urbana integral de Maringá é R$ 5,80 desde 12/07/2026, conforme notícia da Prefeitura documentada no sistema. Não generalize para Sarandi ou benefícios. O saldo da carteira não é um bilhete. Não afirme ligação ao GPS da frota, bilhetagem real ou previsão de chegada.